

JMLR MLOSS re-review: kodama-cpp

Assessment after revision of the evidence and reproducibility claims

Review date: 8 August 2026

Summary

This submission presents a substantial standalone implementation of KODAMA rather than a new learning objective. The manuscript clearly distinguishes continuity with the published KODAMA method from the new C++17 systems contribution: float32 state ownership, CPU/CUDA/Metal backends, label-aware SIMPLS plus latent-space LDA, package-owned neighbor search, supplied-graph entry points, and thin R/Python bindings. The algorithm is reproducible from the equations and pseudocode: proposal generation, one-CV-pass-per-cycle evaluation, acceptance score, cooling rule, independent M runs, landmark and splitting semantics, projection, and agreement-graph correction are all specified.

The empirical section now includes paired three-seed classic-versus-KODAMA CUDA results across an 11-dataset archive. The dataset is the inferential unit. For PLS-LDA plus UMAP, 6 of 10 complete datasets improve, the median silhouette change is +0.025, the 100,000-resample bootstrap interval is [-0.067, +0.084], and the Holm-adjusted exact Wilcoxon p-value is 0.557. KNN plus openTSNE is worse on all 10 datasets (Holm-adjusted p=0.0078). These post-specified analyses are labeled exploratory; the manuscript does not claim universal visualization improvement.

A new four-panel ImageNet figure makes the software output concrete, and the supplement now reports preliminary KODAMA core timing for every completed dataset/classifier cell. The figure is appropriately described as illustrative: its archived classic and KODAMA renders do not yet document one identical plotting-index vector. The frozen rerun plan requires matched plotting indices and fills the missing FlowRepository KNN and ImageNet PLS-LDA timing cells.

A local Apple Metal scheduler study additionally shows that the optimal number of independent worker lanes depends on the retained fold workspace: three lanes were selected for raw COIL20 and four for MetRef. This supports memory-aware automatic scheduling, but the timing is preliminary because thermal state and run order were not controlled as rigorously as the planned confirmatory experiment.

A new paired CPU4/Metal screen removes graph-search confounding by supplying one identical CPU graph to both optimizers for each dataset and seed. Across three seeds at M=20/Tcycle=20, median Metal speedups range from 0.98x for MetRef PLS-LDA to 2.33x for USPS PLS-LDA. The study properly retains the raw COIL20 50-component PLS-LDA cell as censored after it crossed the local feasibility budget. The wrapper now also invalidates stale native objects when the selected core archive or public headers change. These are useful engineering revisions, but the results remain dirty-worktree evidence.

Two further local corrections strengthen the numerical implementation. CPU PLS-LDA now streams class sums and the latent Gram matrix instead of retaining full score matrices; TabulaMuris PLSLDACV improved by 1.47x with identical predictions and a 0.587 peak-memory ratio. An overflow-only SIMPLS power-norm guard restored all 50 requested components on MNIST70k: CPU accuracy changed from an erroneous 0.112529 fallback to 0.866457, while Metal changed from failure to 0.866286 in a 2.296-second median. Normal CPU, AddressSanitizer, physical Metal, and a compact synthetic regression pass. Frozen CUDA confirmation is still required.

Cycle 15 adds a full shared-graph MetRef run at M=Tcycle=100 and clean wrapper checks. Metal accelerated KNN optimization from 2.729 to 1.609 seconds, but four-lane Metal PLS-LDA required 651.767 seconds versus 335.734 seconds on CPU4. Both PLS-LDA backends reached median CV accuracy 0.993893 and strong truth-label silhouette (0.8598/0.8656), while KNN selected two classes. A single-lane Metal proposal was rejected after two runs required 46.8 seconds. This strengthens the paper because it reports an unfavorable systems result and avoids a benchmark-specific scheduler rule.

Cycle 16 adds the corresponding full shared-graph USPS controls. Metal accelerated KNN optimization from 12.451 to 7.888 seconds, but PLS-LDA required 1105.998 seconds on Metal versus 707.057 seconds on CPU4. KODAMA truth-label silhouettes were 0.4550/0.4555 for KNN and 0.4227/0.4174 for PLS-LDA, compared with approximately 0.13 for classic UMAP. The result reinforces the need for stage-resolved, classifier-specific performance claims.

Cycle 22 adds a complete shared-graph SatImage control at $M=T_{\text{cycle}}=100$. Metal was 1.06x slower than CPU4 for KNN and 5.87x slower for PLS-LDA; both KODAMA truth-label silhouettes were below classic UMAP. The same cycle raises CPU line/branch coverage to 80.35%/68.79% by testing the public fastEmbedR-derived binary UMAP graph route, including edge compaction, non-default curve fitting, fixed-seed replay, and failure when no usable edge remains. Production mathematics is unchanged.

Disposition of Previous Comments

Comment	Status	Revision or remaining action
Uncertainty and paired inference	Addressed	Dataset-level seed medians, bootstrap intervals, exact paired Wilcoxon tests, sign tests, and Holm adjustment are now reported.
Primary endpoint	Addressed for a future confirmatory study	PLS-LDA UMAP silhouette is predeclared as primary; trustworthiness and Preserve@30 are secondary. Existing archive analyses remain explicitly exploratory.
Overstatement of silhouette	Addressed	The text now describes reference-label compactness and label-aligned structure, not recovery of more information.
Unsupported Metal timing	Addressed	The historical 425x row was removed because warm-up and synchronization boundaries were not retained.
Timing boundaries	Addressed	The protocol defines wall-clock call boundaries, synchronization, allocation and transfer inclusion, warm-up handling, and thread reporting.
Release identity and HPC provenance	Open	Public commit identities are recorded, but no immutable tag exists and uploaded manifests contain <code>git_commit=NA</code> ; a clean tagged confirmatory rerun is still required.
Coverage and CUDA audit	Partially addressed	A direct public-entry-point map records physical Metal and current-source CUDA execution. Binary UMAP contracts raise measured CPU line/branch coverage to 80.35%/68.79%, but this remains below the MLOSS target and public CI still does not exercise CUDA hardware.
Independent adoption	Open	Predecessor-package use is documented separately, but the new library still lacks verifiable independent adoption evidence.
Standalone Python repository	Open	The tested binding remains embedded in the core tree rather than published as the stated independent wrapper repository.
Metal worker scheduling	Partially addressed	Automatic lanes now use an implementation-specific fold-workspace estimate; randomized, thermally controlled replication and the matched CUDA experiment remain open.

Comment	Status	Revision or remaining action
Backend-comparison confounding	Addressed locally	A shared-graph mode now feeds one identical graph to CPU4 and Metal and is reported separately from native end-to-end timing; the tagged CUDA counterpart remains open.
R wrapper stale relinking	Addressed locally	A checksum signature over the selected archive and public headers forces both Rcpp objects and the shared library to rebuild after a core change; unchanged installs remain incremental.
CPU PLS-LDA score storage	Addressed locally	The CPU path streams class sums and latent Gram statistics; four-dataset predictions match and the large-n case is faster and smaller.
Metal PLS-LDA score transfers	Addressed locally	Training scores remain device-resident, a label-aware kernel forms class sums, MPS forms T'T, validation scoring is on-device, and five-dataset tests retain the small-data crossover.
Large-sample SIMPLS range	Addressed locally	MNIST70k and a compact overflow regression pass on CPU/Metal; the frozen CUDA confirmation remains open.
Full-cycle local wrapper evidence	Addressed locally	R CMD check passed with no errors or warnings and one new-submission NOTE; an installed-core Python wheel passed eight CPU/physical-Metal tests; full MetRef, USPS, ImageSegmentation, PageBlocks, PenDigits, SatImage, and paired COIL20 controls are retained. Landmark timing is exposed separately.

Major Comments

1. The reviewed software is still a release candidate rather than an immutable reviewed release. There is no public v0.1.0 tag or GitHub release, and no archived source snapshot. Freeze the exact reviewed commit, create the source archive required by MLOSS, publish its checksum, and cite that identity consistently. A DOI is useful but not itself mandatory.
2. The new HPC archive is broad, but its KODAMA run manifests record git_commit as NA. Therefore the central 11-dataset result cannot yet be tied unambiguously to the reviewed source. Reproduce or formally identify these runs from a tagged core and wrapper commit, archive the benchmark configuration and hardware record, and preserve the same seed-level CSV files.
3. The ImageNet panel is useful, but the archived KODAMA render explicitly uses a fixed 250,000-point plotting subset while the retained classic plot does not record the same index vector. Keep the current figure qualitative. In the definitive rerun, save one stratified plotting-index vector and apply it, the same class palette, point size, axis treatment, and rasterization policy to all four layouts.
4. The all-dataset timing table is preliminary and incomplete: FlowRepository KNN and ImageNet PLS-LDA are absent, and the archive is not tied to a release commit. Complete both cells or report a predeclared resource-limit failure, then provide graph, optimization, projection, correction, visualization, total wall time, host RAM, and device memory from the same tagged benchmark protocol.
5. CPU source coverage is 80.35% by line and 68.79% by branch, still below the MLOSS expectation of coverage close to 100%. Public binary UMAP, matrix orchestration, invalid-input, unavailable-backend, fixed-seed replay, monotone best-trace, guarded one-class, anti-collapse, shake-recovery, and constant-predictor PLS-LDA fallback

contracts are now exercised. Add further tests for resident-IVF lifecycle, graph materialization, matrix/visualization integration, and accelerator failure paths. CUDA is not exercised by public CI; archive hardware logs or add a maintained GPU runner.

6. The cover letter documents substantial use of the predecessor KODAMA R package, but this is not evidence of an active community around kodama-cpp. The new repository currently has no stars, forks, releases, public issues, or independently documented adopters. Obtain and cite verifiable beta-user, downstream-package, issue, or external-reproduction evidence rather than transferring predecessor adoption to the new implementation.
7. The tested Python binding is public only as an embedded source subtree, while the project architecture and submission materials describe independently versioned wrappers. Publish the standalone Python repository before submission, test installation from that public source in a clean environment, and state clearly which wrapper commit belongs to the reviewed core release.
8. Raw high-dimensional PLS-LDA is now characterized at kernel level: the COIL20 predictor-width study demonstrates a CPU/Metal crossover and exposed a stale-fold residency bug that is now fixed and regression-tested. Metal also keeps projected scores resident, uses a label-aware class-sum kernel and MPS T'T, and scores validation rows on-device. In the current full protocol, raw COIL20 CPU4 at M=Tcycle=100 exceeded 300 seconds before the first four independent runs completed and is explicitly censored. Complete release-tagged CPU4/CUDA scaling with the newly separated landmark timing, peak memory, and the distinct CUDA cached-Gram versus Metal retained-matrix formulations is still required.
9. The robust SIMPLS norm fixes a severe local MNIST70k failure and is supported by replicated CPU/Metal results, but it has not been exercised on the frozen CUDA artifact. Run fixed-50-component MNIST70k PLSLDACV and the compact overflow regression on tagged CUDA, record selected components and predictions, and retain any bounded float32 difference rather than assuming cross-backend parity.

Minor Comments

1. Complete the author-only cover-letter fields: coauthor consent, funding, competing interests, and conflict-free editor/reviewer suggestions. Refresh all dated repository and download metrics immediately before submission.
2. Use KODAMA R 2.4.1/2.4 as the only predecessor timing baseline. Keep the comparison classifier-matched where possible, use M=100/Tcycle=100, and state explicitly that revised proposal dynamics prevent trajectory-level parity.
3. Document the native CUDA graph-builder limit $k \leq 256$ prominently and report unsupported requested graph sizes as missing rather than silently truncating them.
4. Retain the explicit distinction between internal CV accuracy, the search score, and external-label diagnostics. The new visualization table makes the non-equivalence especially clear and should not be simplified into a claim that maximizing CV accuracy necessarily recovers reference classes.
5. Clarify that classic rows are full fastEmbedR package calls whereas KODAMA rows use the library's pinned standalone embedding port on a corrected graph. The mathematical lineage and parameters are matched, but the software-call boundaries and total runtimes are intentionally different.
6. Keep the four-page paper self-contained while treating the long technical document as a supplement. The present layout now satisfies the four-description-page limit, with references beginning on page 5.
7. The current technical supplement is comprehensive but long. Remove duplicated narrative, retain auditable implementation details and complete tables, and move operational logs into the release archive.
8. The inherited CUDA helper symbols and environment controls now use KODAMA-owned prefixes and a source/link audit prevents regression. Close the remaining accelerator-specific part by inspecting the tagged CUDA symbol table and loading kodama-cpp beside fastEmbedR in one process.
9. Keep KODAMA R 2.4.1/2.4 as the sole predecessor. The retained 16 July table and a later 6 August diagnostic have different single-run timings; identify the final tagged protocol unambiguously and report replicated within-host medians rather than combining or selecting between those preliminary rows.

Strengths

- The distinction between raw CV accuracy, proposal acceptance score, and external diagnostics is unusually clear and prevents circular quality claims.

- The single-run and ensemble pseudocode make the stochastic optimization independently implementable.
- Backend identity is strict and testable; unavailable accelerators do not masquerade as successful CPU execution.
- The evidence matrix links implementation claims to source locations, tests, and measured results.
- Cross-platform GitHub Actions, measured CPU coverage, a frozen API test, provenance audit, and hosted documentation substantially strengthen MLOSS readiness.
- The manuscript retains unfavorable datasets and now quantifies that KODAMA correction is classifier-, embedding-, and dataset-dependent.
- The paired HPC analysis spans image, cytometry, metabolomics, and single-cell representations and avoids using unmatched runs in comparative summaries.

Reviewer Recommendation

Recommendation: Major Revision. The software is suitable in scope and technically promising for JMLR MLOSS, and the four-page description now contains a substantive multi-dataset result. However, the reviewed artifact is not yet frozen, the new HPC runs lack a recorded git commit, ImageNet plotting indices are not yet matched across all four panels, the timing matrix has two missing cells, coverage remains substantially below the track's stated expectation, active adoption of the new library has not yet been demonstrated, complete high-dimensional PLS-LDA pipeline scaling is not yet characterized, the CUDA scheduler and robust-norm experiments remain outstanding, and the standalone Python repository described by the architecture is not public. The earlier statistical-interpretation concern is now adequately addressed for an exploratory study.

The full MetRef, USPS, ImageSegmentation, PageBlocks, PenDigits, and SatImage results, plus paired COIL20 KNN and censored raw PLS-LDA, do not change this recommendation. They close a useful local full-cycle gap, but also demonstrate that native Metal availability is not a universal acceleration claim: Metal can accelerate KNN while CPU4 is faster for complete PLS-LDA in several matrix regimes. The main text and supplement must preserve this classifier- and workload-specific interpretation. PenDigits, COIL20, and SatImage further show that high internal CV accuracy can coexist with a degraded final visualization.

I would encourage resubmission after these software-release and evidence issues are addressed. I do not request a new KODAMA objective or an algorithmic redesign: the principal work is to freeze and test the release, bind the HPC archive to that release, strengthen coverage and accelerator auditability, publish the Python wrapper, and establish independent usage evidence.